Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Proses Senkronizasyonu (Process Synchronization) Giriş
  - Process'ler ortak kaynaklara (global değişken, veri yapısı, dosya) eşzamanlı eriştiğinde tutarsızlıklar oluşabilir.
  - Senkronizasyon, paylaşılan kaynaklara erişimi düzenleyerek tutarlılık sağlar.
  - İki temel kavram: Critical Section (Kritik Bölge) ve Race Condition (Yarış Durumu).
- Critical Section (Kritik Bölge)
  - Bir process'in paylaşılan kaynağa erişip değişiklik yaptığı bölge.
  - Örnek: `counter++` operasyonu aslında 3 instruction'dan oluşur (read, modify, write).
  - Process kritik bölgede iken kesilirse (preempt) ve başka process aynı veriye erişirse, tutarsızlık oluşur.
  - Çözüm: Process kritik bölgede iken kesintiye uğramamalı (mantıksal olarak).
- Race Condition (Yarış Durumu)
  - Birden fazla process aynı veriye eşzamanlı eriştiğinde, sonucun erişim sırasına bağlı olması.
  - Örnek: counter=5 iken P1 ve P2 aynı anda counter'ı arttırıp eksiltmek isterse, sonuç 5 (P1 önce), 4 (P1 sonra, P2 önce) veya 3 (P2 sonra) olabilir.
  - Her çalıştırmada farklı sonuç çıkabilir; tanımlı değil.
- Mutual Exclusion (Karşılıklı Dışlama)
  - Bir process kritik bölgede iken başka process'ler o bölgeye giremez.
  - 4 şart sağlanmalı: Mutual exclusion, Progress, Bounded waiting, No preemption (kesintisiz çalışma).
- Peterson Çözümü (Yazılımsal)
  - İki process arasındaki karşılıklı dışlama için yazılımsal çözüm.
  - `flag[]` dizisi: Her process kritik bölgeye girmek isteğini belirtir.
  - `turn` değişkeni: Sıranın kimde olduğunu gösterir.
  - Basit ama ölçeklenebilir değil; modern sistemlerde donanım desteği kullanılır.
- Donanımsal Çözümler
  - **Memory Barriers (Bellek Engelleri)**: Memory modeline uygun sıralama garantisi.
  - **Hardware Instructions (Test-and-Set, Compare-and-Swap)**: Atomik donanım instruction'ları.
    - `test_and_set`: Hedefi 1 yapar, eski değeri döndürür.
    - `compare_and_swap`: Hedef beklenen değere eşitse yeni değerle değiştirir, eski değeri döndürür.
  - Atomik = kesintisiz, tek instruction olarak çalışır.
- Mutex Locks (Karşılıklı Dışlama Kilitleri)
  - Basit kilit mekanizması. `acquire()` ile kilit alınır, `release()` ile bırakılır.
  - Kritik bölgeye girmeden önce `acquire()`, çıkarken `release()`.
  - Dezavantaj: Busy waiting (kilit açılana kadar CPU harcar).
- Semaphores (Semaforlar)
  - Synchronization için kullanılan tamsayı değerli değişken + bekleme kuyruğu.
  - İki tür:
    - **Binary Semaphore (Mutex)**: Değer 0 veya 1. Karşılıklı dışlama için.
    - **Counting Semaphore**: Değer 0-N. Belirli sayıda kaynağa erişim için.
  - İki operasyon:
    - `wait()` (P): Değer ≤ 0 ise bloklan; > 0 ise azalt.

    * `signal()` (V): Değeri arttır, bekleyen process'i uyandır.
  - Kritik bölgelerde `wait(mutex)` ve `signal(mutex)` ile korunur.
- Senkronizasyon Problemleri
  - **Bounded-Buffer (Sınırlı Tampon) Problem**: Producer-consumer. 3 semafor: `mutex` (1), `full` (0), `empty` (N).
  - **Readers-Writers Problem**: Birden fazla okuyucu aynı anda okuyabilir, ama yazıcı tek başına çalışmalı. `rw_mutex` ve `mutex` semaforları.
  - **Dining-Philosophers Problem**: 5 filozof, 5 çubuk (chopstick). Her filozofun yemek yemesi için 2 çubuk gerekir. Deadlock olabilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Semafor Değerinin Sıfır Olamaması
  - Hoca vurgular: "Semaforun değeri hiçbir zaman sıfır olamaz diyoruz. Neden? Çünkü sıfır olduğu durumda semafor hiç yok. Ve process'ler semaforun eğer varlığı varsa kaynağa erişebiliyor ya. Integer değeri 1 ise mesela. O 1 değerini gördüğü zaman kaynağa erişebiliyor ve aynı zamanda semaforun değerini sıfıra çekiyor."
- Mesaj Kuyruğunda Unlink Gerekliliği
  - Hoca vurgular: "Mesaj queue unlink fonksiyonuyla ilişkisi kopartılmamış hiçbir kuyruk edilemez. Bu fonksiyon ile sonlandırılamaz demişiz. Unlink fonksiyonu silmek üzere işaretliyorum, işim bitti artık benim bu semaforla deyip artık silecek olan fonksiyona benim işim bitti deyip bildiri veriyorduk."
- Sem_open/Sem_close/Sem_unlink
  - "Bütün process'ler işaretledi benim bu semaforla işim kalmadı diye daha sonra da eminim ki ben process'ler kullanmayacak semaforu. O zaman semclose ile bu semaforu silebilirim."
- Priority-Based Message Queue Okuma
  - Hoca vurgular: "Mesaj kuyruğunu da en öne geçti bu mesaj. Priority'si 110 aldığı için." Yani priority alanı yüksek olan mesaj önce okunur.
- Fork Sonrası Message Queue Aktarılamaz
  - "Fork'la yarattığımız process'lere bu message queue'larımızı aktaramıyoruz." Çünkü mesaj kuyruğu process'in sahip olduğu bir kaynak değildir. Message queue'yu tekrar open etmek gerekir.
- Protection Bayrakları
  - `PROT_READ`: Belleği okuyabilir.
  - `PROT_WRITE`: Belleğe yazabilir.
  - `PROT_EXEC`: Belleği execute edebilir.
  - `PROT_NONE`: Belleğe hiç erişim yok.
  - `MAP_SHARED`: Değişiklikleri paylaş.
  - `MAP_PRIVATE`: Değişiklikleri paylaşma (copy-on-write).
  - `MAP_FIXED`: Adresi sabit.

Kısa Tekrar Notları

- Critical Section = Paylaşılan kaynağa erişim bölgesi.
- Race Condition = Eşzamanlı erişim sonucu belirsizlik.
- Mutual Exclusion = Aynı anda sadece bir process.
- Test-and-Set, Compare-and-Swap: atomik donanım instruction'ları.
- Mutex: binary semaphore.
- Semaphore: tamsayı değer + kuyruk.
- wait(P) azalt veya bloklan, signal(V) arttır veya uyandır.
- Bounded buffer: mutex + full + empty.
- Dining philosophers: deadlock riski, çubuklar.

Detaylı Açıklamalar

Ders 10, proses senkronizasyonunu derinlemesine ele alır. Bu ders, IPC konusunun devamı niteliğindedir: process'ler arası iletişimde veri tutarlılığı nasıl sağlanır?

Process'ler ortak kaynaklara (global değişken, veri yapısı, dosya) eriştiğinde tutarsızlıklar oluşabilir. Örnek: iki process aynı `counter` değişkenini arttırmak istiyor. Her biri `counter++` yapar. Bu operasyon aslında 3 instruction'dan oluşur: register'a oku (read), register'ı arttır (modify), register'ı belleğe yaz (write). Eğer P1 read yaptıktan sonra preempt edilir ve P2 read-modify-write yaparsa, P1 modify-write yaptığında eski değer üzerine yazılmış olur; bir arttırma kaybolur.

Race condition, birden fazla process aynı veriye eşzamanlı eriştiğinde sonucun erişim sırasına bağlı olmasıdır. Her çalıştırmada farklı sonuç çıkabilir; bu kabul edilebilir değildir. Senkronizasyon bu sorunu çözmek için gereklidir.

Critical section problemi, bir process'in paylaşılan kaynağa eriştiği kod bölgesinin korunmasıdır. Bir process kritik bölgede iken başka process'ler o bölgeye girememelidir (mutual exclusion). Bu problemi çözmek için çeşitli yaklaşımlar vardır:

Peterson Çözümü: İki process arasındaki karşılıklı dışlama için yazılımsal çözümdür. `flag[]` dizisi (her process kritik bölgeye girme isteğini belirtir) ve `turn` değişkeni (sıranın kimde olduğu) kullanılır. Basit ama ölçeklenebilir değildir.

Donanımsal Çözümler: Modern işlemciler atomik (kesintisiz) instruction'lar sağlar. `test_and_set` hedefi 1 yapar ve eski değeri döndürür. `compare_and_swap` üç parametre alır: hedef, beklenen değer, yeni değer; eşitse değiştirir. Bu instruction'lar atomiktir, kesilmez.

Mutex Locks: Basit kilit mekanizması. `acquire()` ile kilit alınır, `release()` ile bırakılır. Dezavantajı: busy waiting (kilit açılana kadar CPU harcamak). Bu nedenle modern sistemlerde semaforlar tercih edilir.

Semaphores: Senkronizasyon için en yaygın kullanılan yapıdır. Tamsayı değer ve bekleme kuyruğundan oluşur. İki operasyonu vardır: `wait(P)` (değer ≤ 0 ise bloklan, > 0 ise azalt) ve `signal(V)` (değeri arttır, bekleyen process'i uyandır). Binary semaphore (mutex) değeri 0 veya 1; counting semaphore değeri 0-N olur.

Üç klasik senkronizasyon problemi detaylı şekilde anlatılır:

Bounded-Buffer (Producer-Consumer): 3 semafor kullanılır: `mutex` (1, karşılıklı dışlama), `full` (0, dolu slot sayısı), `empty` (N, boş slot sayısı). Producer: `wait(empty)` - `wait(mutex)` - üret - `signal(mutex)` - `signal(full)`. Consumer: `wait(full)` - `wait(mutex)` - tüket - `signal(mutex)` - `signal(empty)`. full + empty = N her zaman.

Readers-Writers: Veritabanı problemine benzer. Birden fazla okuyucu aynı anda okuyabilir, ama yazıcı tek başına çalışmalı. `readcount` (okuyucu sayısı), `rw_mutex` (yazıcılar için), `mutex` (readcount koruması).

Dining-Philosophers: 5 filozof, 5 çubuk. Her filozof düşünür veya yer. Yemek için 2 çubuk gerekir. Klasik çözüm deadlock'a yol açar. Asimetrik çözüm: tek numaralı filozoflar önce sol, çift numaralılar önce sağ çubuğu alır. Bu da kaynak dengesizliğine yol açar. Daha iyi çözüm: sırayı değiştirmek.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.